PAML Spring 2026 Final Project Proposal

Baili Chen
Cornell Tech ORIE
New York, NY
bc722@cornell.edu

Xiaohui Zang
Cornell Tech ORIE
New York, NY
xz973@cornell.edu

Ziyue Zhang
Cornell Tech ORIE
New York, NY
zz893@cornell.edu

Jiayin Zhuang
Cornell Tech ORIE
New York, NY
jz2453@cornell.edu

PAML Spring 2026 Final Project Proposal

***Abstract*—This project focuses on predicting whether a customer will subscribe to a term deposit in a bank marketing campaign. Accurate prediction can improve marketing efficiency and resource allocation. We develop and compare two machine learning models, Logistic Regression and Naive Bayes, where Logistic Regression serves as a baseline for comparison. The models are trained and evaluated using the Bank Marketing dataset from the UCI Machine Learning Repository, which contains demographic, financial, and campaign-related features. Model performance is assessed using accuracy, precision, recall, and F1-score. To support practical use, we propose a Streamlit-based application that provides real-time predictions based on user input. The best-performing model and decision threshold will be selected for deployment. The project aims to demonstrate how machine learning can support data-driven decision-making in financial marketing and improve customer targeting.**

***Index Terms*—Machine Learning, Classification, Logistic Regression, Naive Bayes, Bank Marketing, Streamlit**

I. Introduction

Problem and Importance. In financial marketing, many contacted customers do not subscribe to term deposits, leading to inefficient resource allocation and increased operational costs. Accurately identifying high-potential customers is therefore essential for improving campaign effectiveness and reducing unnecessary outreach.

Overview of Existing Work. Customer subscription prediction has been widely studied using both interpretable models such as Logistic Regression and probabilistic models such as Naive Bayes, which are standard baselines for binary classification on structured marketing data.

Gaps in Prior Work. However, prior work often focuses on predictive performance without fully addressing class imbalance and the impact of decision threshold selection on real-world deployment outcomes.

Proposed Approach. We formulate this task as a supervised binary classification problem using the Bank Marketing dataset and implement Logistic Regression and Naive Bayes from scratch. Model performance is evaluated using accuracy, precision, recall, and F1-score, with emphasis on recall. The best-performing model and decision threshold are deployed in a Streamlit application that enables real-time prediction for marketing users.

Broader Impact. This project demonstrates how interpretable machine learning models can be integrated into practical decision-making systems, improving accessibility for non-technical users and supporting data-driven marketing strategies.

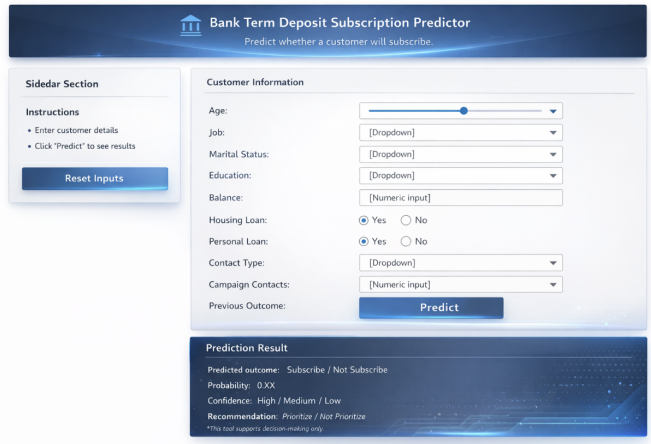

Fig. 1. Wireframe of the Streamlit-based application for predicting customer subscription to term deposits, showing input fields, prediction button, and output panel.

II. Background

Comparisons to Prior Work. Logistic Regression is a widely used method for binary classification due to its interpretability and well-understood statistical properties [1], while Naive Bayes provides a probabilistic alternative based on conditional independence assumptions [2]. These models represent discriminative and generative approaches, respectively.

Common Knowledge and Challenges. A major challenge in customer subscription prediction is class imbalance, where non-subscribers significantly outnumber subscribers. In such settings, accuracy alone can be misleading, and metrics such as precision, recall, and F1-score are required for balanced evaluation [3], [4].

Knowledge Gaps. While existing work explores these models, less attention has been given to linking evaluation metrics with deployment decisions, particularly the role of threshold tuning in balancing business trade-offs. This project addresses this gap by integrating model comparison, metric-based evaluation, and threshold selection into a deployable system.

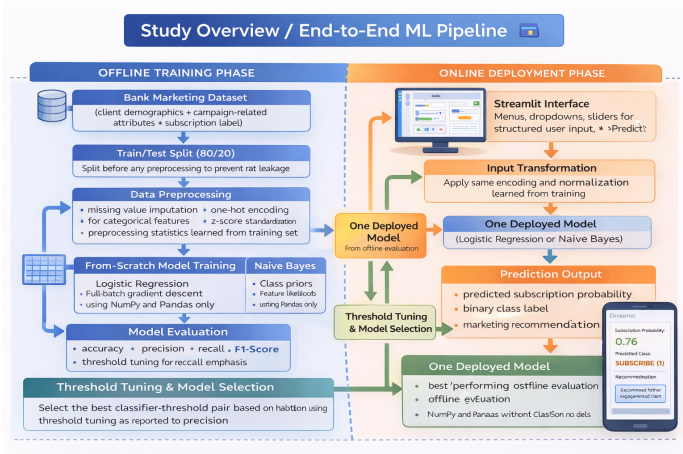

Fig. 2. Overview of the proposed end-to-end machine learning pipeline, including offline training, followed by deployment in a Streamlit interface.

III. End-to-End ML Pipeline

A. Back-End

*1) **Data Collection, Exploration Processing**:*

*a) **Dataset Description.**:* We use the Bank Marketing dataset from the UCI Machine Learning Repository, which contains approximately 45,000 records from a Portuguese banking institution. The dataset includes both numerical and categorical features, such as demographic, financial, and campaign-related variables. The target is a binary label indicating whether a client subscribes to a term deposit.

*b) **Data Usage.**:* The dataset is used for supervised learning, where the labels serve as ground truth. It is split into training and test sets for unbiased model evaluation.

*c) **Data Exploration.**:* Exploratory data analysis includes histograms for numerical features, bar plots for categorical variables, and correlation analysis. Class distribution analysis reveals imbalance between subscription and non-subscription cases, which informs later modeling decisions.

*d) **Preprocessing Techniques.**:* Missing values are handled through imputation, and categorical variables are encoded using one-hot encoding. Numerical features are standardized using z-score normalization. Outliers are examined and mitigated where necessary, and features that may cause data leakage are excluded.

*e) **Justification.**:* These preprocessing steps ensure compatibility with machine learning models and improve training stability. Insights from data exploration guide feature selection and motivate the use of evaluation strategies such as precision-recall trade-offs and threshold tuning.

*2) **Methods and Model Training**:*

Model 1: Logistic Regression

*a) **Problem Formulation.**:* The prediction task is formulated as a supervised binary classification problem. Each observation corresponds to a single bank client and is represented as a numerical feature vector $\mathbf{x} \in \mathbb{R}^d$ obtained after the preprocessing stage described in Section 4.1.1. These feature vectors include both numerical attributes and encoded categorical variables derived from client demographics and campaign interactions. The model must output (i) a predicted probability $\hat{y} \in (0,1)$ that the client will subscribe to a term deposit, and (ii) a binary decision label $\tilde{y} \in \{0,1\}$ derived from that probability via a configurable decision threshold.

*b) **Model Formulation.**:* Logistic regression is selected as the baseline classifier for this project for several reasons. First, it is a well-established and theoretically well-understood method for binary classification on structured tabular data. Second, its learned weight vector $\mathbf{w}$ provides directly interpretable coefficient estimates that relate individual input features to the log-odds of the positive class, which is valuable in a marketing analytics context where feature-level interpretability aids business decision-making. Third, the model is computationally efficient and trains reliably on datasets of the size used here. Fourth, it serves as a natural performance reference point against which the second model in this project can be compared.

Given an input vector $\mathbf{x}$, the model computes a linear score

$$z = \mathbf{w}^\top \mathbf{x} + b, \tag{1}$$

where $\mathbf{w} \in \mathbb{R}^d$ is the weight vector and $b \in \mathbb{R}$ is the scalar bias term. The linear score is passed through the sigmoid activation function

$$\sigma(z) = \frac{1}{1+e^{-z}}, \tag{2}$$

which maps z to the interval $(0,1)$, yielding the predicted probability

$$\hat{y} = \sigma\left(\mathbf{w}^\top \mathbf{x} + b\right). \tag{3}$$

*c) **Loss Function.**:* The model is trained by minimising the binary cross-entropy loss over the training set of n examples:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right]. \tag{4}$$

To prevent numerical overflow, predicted probabilities are clipped to the interval $[\epsilon,\, 1-\epsilon]$ with $\epsilon = 10^{-15}$ before the logarithm is applied.

*d) **Training Procedure.**:* The model is trained using full-batch gradient descent. The weight vector $\mathbf{w}$ is initialised to the zero vector and the bias b is initialised to zero. At each iteration the gradient of the loss with respect to $\mathbf{w}$ and b is derived analytically from the binary cross-entropy objective:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{1}{n}\mathbf{X}^\top(\hat{\mathbf{y}} - \mathbf{y}), \qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i), \tag{5}$$

where $\mathbf{X} \in \mathbb{R}^{n\times d}$ is the design matrix of training examples. Parameters are updated as

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha\frac{\partial \mathcal{L}}{\partial \mathbf{w}}, \qquad b \leftarrow b - \alpha\frac{\partial \mathcal{L}}{\partial b}, \tag{6}$$

where α is the learning rate. The entire implementation is written from scratch in Python using NumPy and Pandas only; no high-level machine learning frameworks such as scikit-learn are used at any stage of the modelling pipeline.

*e) **Implementation Details.**:* The logistic regression model is encapsulated in the class `LogisticRegressionScratch`. The class exposes the method `_sigmoid` for computing the sigmoid activation and `_binary_cross_entropy` for computing the scalar training loss at each iteration. Training is invoked through the `fit` method, which accepts a pre-processed feature matrix and a binary label vector and executes gradient descent for a fixed number of iterations. The model is instantiated with a learning rate of $\alpha = 0.01$ and trained for 1,000 iterations. After training, continuous posterior probabilities $P(y = 1 \mid \mathbf{x})$ are obtained via `predict_proba`, while binary class predictions are produced by `predict`, which applies a user-specified decision threshold (defaulting to 0.5) to the output of `predict_proba`.

*f) **Data Leakage Prevention.**:* The dataset is partitioned into an 80% training set and a 20% test set prior to any preprocessing operation. All preprocessing statistics—including feature means and standard deviations used for z-score standardisation, imputation values for missing entries, and one-hot encoding column mappings—are estimated exclusively from the training partition and subsequently applied to both the training and test sets without modification. This design strictly prevents information from the test set from influencing the learned preprocessing parameters, ensuring that all reported evaluation metrics reflect true out-of-sample generalization performance.

Model 2: Naive Bayes

*g) **Problem Formulation.**:* Similar to the logistic regression baseline, the Naive Bayes model is applied to the same supervised binary classification task. Each client is represented as a feature vector $\mathbf{x} \in \mathbb{R}^d$ derived from the preprocessing pipeline, and the model outputs a probability $\hat{y} \in (0, 1)$ indicating the likelihood of subscription, along with a binary classification decision obtained via a threshold.

*h) **Model Formulation.**:* In addition to the logistic regression baseline, we implement a Naive Bayes classifier as a second model for comparison. Naive Bayes is a probabilistic model based on Bayes' Theorem and assumes conditional independence among input features given the class label. Given an input vector $\mathbf{x} = (x_1, x_2, \ldots, x_d)$, the model estimates the posterior probability:

$$P(y \mid \mathbf{x}) \propto P(y) \prod_{i=1}^{d} P(x_i \mid y), \tag{7}$$

where $P(y)$ is the prior probability of class y, and $P(x_i \mid y)$ represents the likelihood of feature x_i given the class. The predicted class is obtained by selecting the class with the highest posterior probability.

*i) **Assumptions and Limitations.**:* A key assumption of Naive Bayes is the conditional independence of features, which may not strictly hold in real-world datasets such as the Bank Marketing data. However, despite this simplifying assumption, the model often performs well in practice and provides a useful benchmark for comparison.

*j) **Suitability.**:* Naive Bayes is computationally efficient and scalable to large datasets. It can effectively handle high-dimensional feature spaces and is particularly well-suited for datasets containing a mix of categorical and numerical variables. These properties make it a practical candidate for this application.

*k) **Comparison with Baseline.**:* In contrast to logistic regression, which is a discriminative model that directly learns decision boundaries, Naive Bayes is a generative model that models the joint distribution of features and labels. This distinction enables a meaningful comparison between two different modeling paradigms and helps assess whether a simpler probabilistic model can achieve competitive performance.

*l) **Implementation Details.**:* The Naive Bayes classifier is implemented from scratch using NumPy and Pandas, consistent with the project constraints. Model parameters, including class priors and feature likelihoods, are estimated solely from the training data, ensuring alignment with the data leakage prevention strategy applied to the logistic regression baseline.

*3) **Model Evaluation**:*

We evaluate the models using accuracy, precision, recall, and F1-score derived from the confusion matrix. These metrics are chosen to provide a comprehensive view of performance, especially under class imbalance, where accuracy alone may be misleading. In particular, recall is emphasized to capture the model's ability to identify positive cases. The dataset is split into 80% training and 20% test sets. We analyze threshold sensitivity to balance the trade-off between precision and recall, adjusting the decision threshold based on business objectives. This allows the model to be tuned toward either minimizing false positives or false negatives. Hyperparameters are tuned experimentally to improve performance and robustness. Overfitting is mitigated through strict train/test separation and consistent preprocessing across datasets, ensuring that evaluation results reflect true generalization ability.

*4) **Model Deployment**:*

The selected model is deployed as a lightweight decision-support tool using a Streamlit application. The system is designed for marketing teams to support campaign targeting decisions. The application takes user-level features (e.g., demographic and campaign-related variables) as input, applies the same preprocessing pipeline, and generates predictions in real time. For each input, the model outputs both the predicted probability of subscription and a binary classification based on a configurable threshold. This design enables users to adjust the decision threshold based on business objectives, such as prioritizing higher recall to identify more potential subscribers. By maintaining consistency between training and inference, the system ensures reliable and interpretable predictions in practice.

B. Front-End (Streamlit)

*1) **Target Population**:* The primary target population consists of bank marketing teams, sales representatives, and campaign analysts who need to identify high-potential customers for targeted marketing efforts. These users typically do not have expertise in machine learning, so the application is designed to be intuitive and easy to use.

A secondary user group includes students and instructors who may use the application for educational purposes to better understand how machine learning models can be applied in financial marketing and customer behavior prediction.

*2) **User Interface (UI) Layout**:* The application will adopt a structured layout with three main sections:

- Header Section: Displays the application title and a short description of the prediction task

- Sidebar Section: Includes user instructions and a "Reset Inputs" button

- Main Interface: Contains customer input fields and the prediction output panel

The layout is designed to guide users through a simple input-to-prediction workflow.

*3) **User Inputs**:* The application will collect structured inputs from the user, including age, job type, marital status, education, account balance, housing loan, personal loan, contact type, campaign contacts, and previous outcome.

Categorical variables will use dropdown menus, binary variables radio buttons, and numerical values sliders or numeric inputs. This structured design reduces data entry errors and ensures consistency with the preprocessing steps used during model training.

*4) **Expected Outputs**:* The application will display the predicted class label (subscribe / not subscribe), predicted probability score, confidence level, and a recommendation message for marketing follow-up. These outputs are designed to support decision-making rather than provide absolute conclusions. By showing both the classification result and probability estimate, the application gives users a more interpretable understanding of the prediction.

*5) **Front-End and Back-End Integration**:* The front-end is implemented using Streamlit, while the back-end consists of the single selected model trained offline. After entering customer information, the user clicks the "Predict" button, which triggers the following process:

1. Inputs are captured from the interface
2. Inputs are converted into structured feature vectors
3. Preprocessing steps, including encoding and normalization, are applied
4. The processed data is passed into the deployed model
5. The model generates a predicted label and probability score
6. Results are displayed in the output panel

After entering customer information, the user clicks the "Predict" button, which triggers the workflow illustrated in Fig. 1. Inputs are captured from the interface, converted into structured feature vectors, processed using the same encoding and normalization pipeline used during training, passed to the deployed model, and returned as a predicted label and probability score. This event-driven workflow reflects a practical deployment scenario in which user actions directly trigger real-time model inference.

IV. Risk & Mitigation

One challenge is invalid or inconsistent user input, since the application is designed for non-technical users. To reduce this risk, we will use structured inputs such as dropdown menus, radio buttons, sliders, bounded numeric fields, clear labels, and brief instructions. Another challenge is inconsistency between offline training and online deployment, which may cause inaccurate predictions if preprocessing differs. We will address this by reusing the same preprocessing pipeline during deployment. Users may also misinterpret model outputs, so the application will display both the predicted class and probability score with a short disclaimer. Finally, because Logistic Regression and Naive Bayes may perform differently, we will use offline evaluation to choose the deployed model based on recall, interpretability, and usability.

V. Expected Outcomes

We expect both Logistic Regression and Naive Bayes to achieve reasonable classification performance on the bank marketing dataset, with measurable differences across accuracy, precision, recall, and F1-score. We also expect offline evaluation to identify one model as more suitable for deployment, especially when considering recall and business relevance. In addition, we expect the final Streamlit application to make machine learning predictions more accessible to non-technical users through an interactive interface. Overall, the project is expected to produce both a working classification system and a practical example of user-centered machine learning deployment.

VI. Team Member Contribution

A. Technical Components

Jiayin Zhuang (jz2453): frontend development

Baili Chen (bc722): logistic regression, model training, preprocessing pipeline

Ziyue Zhang (zz893): Naive Bayes, data processing, model evaluation, threshold tuning, deployment design

Xiaohui Zang (xz973): system integration, pipeline design, front-end and back-end coordination, and report organization

B. Writing Components

Jiayin Zhuang (jz2453): 4.2, 5, 6

Baili Chen (bc722): 4.1.2,4.1.3,4.1.4, Abstract and Introduction (methods description)

Ziyue Zhang (zz893): 4.1.1, 4.1.2, 4.1.3, 4.1.4, Abstract and Introduction (methods description)overall report editing

Xiaohui Zang (xz973): Introduction, Background, end-to-end ML pipeline, overall report editing

REFERENCES

[1] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*. Wiley, 2013.

[2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[3] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[4] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.